

# CSC1109 – Data (Analytics) at Speed and Scale

Alessandra Mileo

[alessandra.mileo@dcu.ie](mailto:alessandra.mileo@dcu.ie)

Based on “Mining of Massive Datasets” by  
*Jure Leskovec, Anand Rajaraman, Jeff Ullman*  
Stanford University

<http://www.mmds.org>

# Stream Processing and Storm

- Part 1: Stream Mining
  - Concepts (these slides)
  - Bloom Filters (video lecture)
- Part 2: Storm (in class)
  - Introduction to Storm
  - Developing a Storm app

**Note to other teachers and users of these slides:** We would be delighted if you found this our material useful in giving your own lectures. Feel free to use these slides verbatim, or to modify them to fit your own needs. If you make use of a significant portion of these slides in your own lecture, please include this message, or a link to our web site: <http://www.mmds.org>

# Stream Mining concepts

Mining of Massive Datasets

Jure Leskovec, Anand Rajaraman, Jeff Ullman

Stanford University

<http://www.mmds.org>



# New Topic: Infinite Data

## High dim. data

Locality  
sensitive  
hashing

Clustering

Dimensional  
ity  
reduction

## Graph data

PageRank,  
SimRank

Community  
Detection

Spam  
Detection

## Infinite data

Filtering  
data  
streams

Queries on  
streams

Web  
advertising

## Machine learning

SVM

Decision  
Trees

Perceptron,  
kNN

## Apps

Recommen  
der systems

Association  
Rules

Duplicate  
document  
detection

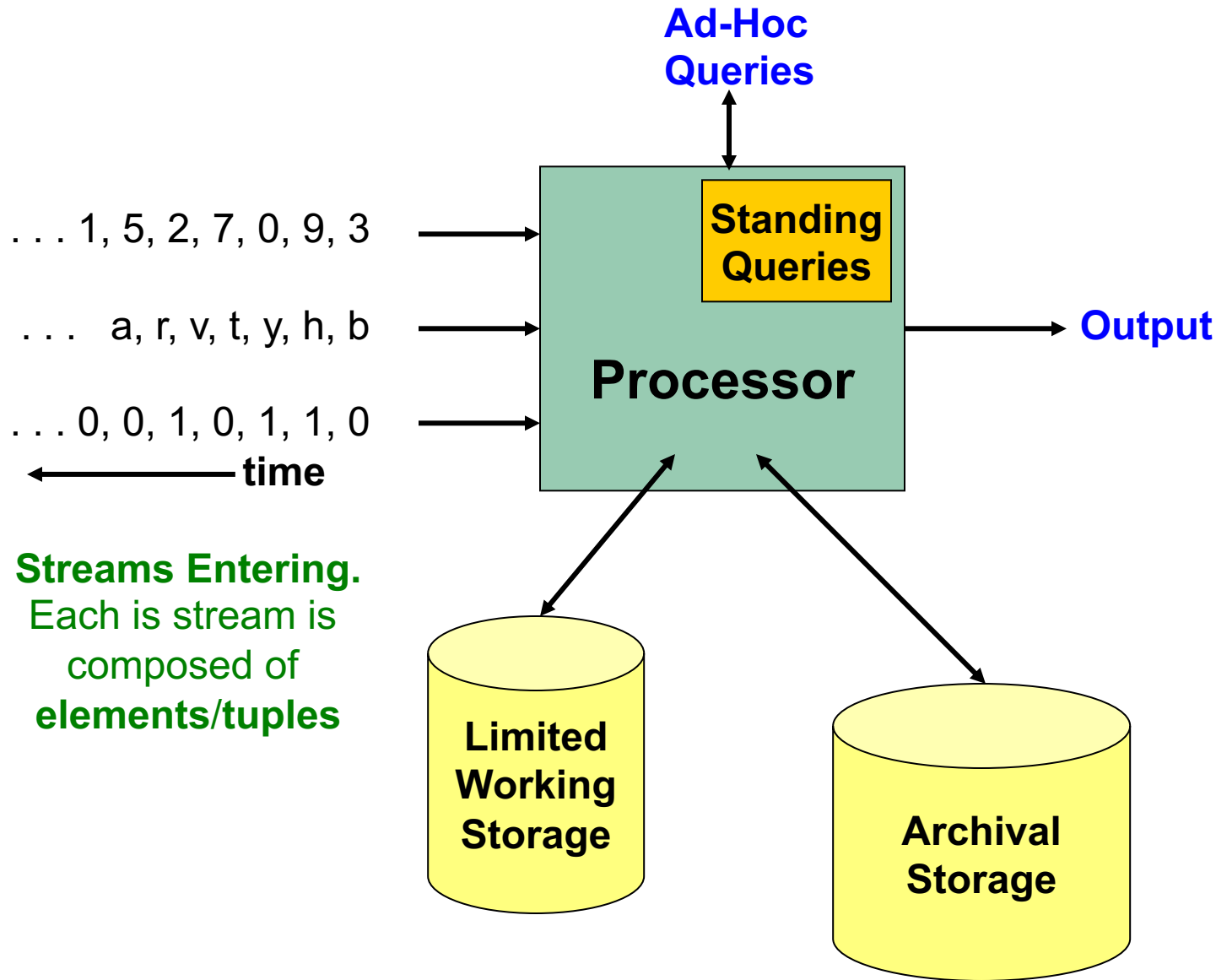
# Data Streams

- In many data mining situations, we do not know the entire data set in advance
- **Stream Management** is important when the input rate is controlled **externally**:
  - Google queries
  - Twitter or Facebook status updates
- We can think of the **data** as **infinite** and **non-stationary** (the distribution changes over time)

# The Stream Model

- Input **elements** enter at a rapid rate, at one or more input ports (i.e., **streams**)
  - **We call elements of the stream tuples**
- **The system cannot store the entire stream accessibly**
- **Q: How do you make critical calculations about the stream using a limited amount of (secondary) memory?**

# Stream Management System



# Applications

- **Mining query streams**
  - Google wants to know what queries are more frequent today than yesterday
- **Mining click streams**
  - Yahoo wants to know which of its pages are getting an unusual number of hits in the past hour
- **Mining social network news feeds**
  - E.g., look for trending topics on Twitter, Facebook



# Queries over a (long) Sliding Window

# Sliding Windows

- A useful model of stream processing is that queries are about a **window** of length  $N$  – the  $N$  most recent elements received
- **Interesting case:**  $N$  is so large that the data cannot be stored in memory, or even on disk
  - Or, there are so many streams that windows for all cannot be stored
- **Amazon example:**
  - For every product  $X$  we keep 0/1 stream of whether that product was sold in the  $n$ -th transaction
  - We want answer queries, how many times have we sold  $X$  in the last  $k$  sales

# Sliding Window: 1 Stream

## ■ Sliding window on a single stream:

**N = 6**

q w e r t y u i o p a s d f g h j k l z x c v b n m

q w e r t y u i o p a s d f g h j k l z x c v b n m

q w e r t y u i o p a s d f g h j k l z x c v b n m

q w e r t y u i o p a s d f g h j k l z x c v b n m

← Past

Future →

# Example: Averages

## ■ Problem:

- Given a stream of integers, window of size  $N$
- Standing query: what is the average of the integers in the window?

## ■ Obvious solution:

- for the first  $N$  inputs, sum and count till you get  $N$  elements, then get the average (and store it)

## ■ Alternative:

- When a new input  $i$  arrives, change the average by adding  $(i-j)/N$ , where  $j$  is the oldest integer in the window